Document number: P0296R2
Date: 2016-6-21
Author: Torvald Riegel <triegel@redhat.com>
Audience: CWG

# P0296R2: Forward progress guarantees: Base definitions

This paper contains wording that clarifies the forward progress requirements in the standard. See P0072R1 for background.

## Wording

Apply the following changes to N4567. First, remove 1.10p3 and add a subheading:

> ~~Implementations should ensure that all unblocked threads eventually make progress. [ Note: Standard library functions may silently block on I/O or locks. Factors in the execution environment, including externally-imposed thread priorities, may prevent an implementation from making certain guarantees of forward progress. --- end note ]~~
>
> **1.10.1 Data races [intro.races]**

Next, change the former 1.10p4 as follows and then move it to after the former 1.10p27:

> Executions of atomic functions that are either defined to be lock-free (29.7) or indicated as lock-free (29.4) are *lock-free executions*.
>
> - If there is only one ~~unblocked~~ thread **that is not blocked (see 17.3.2 [defns.block]) in a standard library function**, a lock-free execution in that thread shall complete. [ Note: Concurrently executing threads may prevent progress of a lock-free execution. For example, this situation can occur with load-locked store-conditional implementations. This property is sometimes termed obstruction-free. --- end note ]
> - When one or more lock-free executions run concurrently, at least one should complete. [ Note: It is difficult for some implementations to provide absolute guarantees to this effect, since repeated and particularly inopportune interference from other threads may prevent forward progress, e.g., by repeatedly stealing a cache line for unrelated purposes between load-locked and store-conditional instructions. Implementations should ensure that such effects cannot indefinitely delay progress under expected operating conditions, and that such anomalies can therefore safely be ignored by programmers. Outside this International Standard, this property is sometimes termed lock-free. --- end note ]

Add a subheading before the former 1.10p27

> **1.10.2 Forward progress [intro.progress]**

Add the following paragraphs before the former 1.10p28:

> **During the execution of a thread of execution, each of the following is termed an *execution step*:**
>
> - **termination of the thread of execution,**
> - **access to a volatile object, or**
> - **completion of a call to a library I/O function, a synchronization operation, or an atomic operation.**
>
> **An invocation of a standard library function that blocks (see 17.3.2 [defns.block]) is considered to continuously execute execution steps while waiting for the condition that it blocks on to be satisfied. [Example: A library I/O function that blocks until the I/O operation is complete can be considered to continuously check whether the operation is complete. Each such check might consist of one or more execution steps, for example using observable behavior of the abstract machine. --- end example]**
>
> **[Note: Because of this and the preceding requirement regarding what threads of execution have to perform eventually, it follows that no thread of execution can execute forever without an execution step occuring. --- end note]**
>
> **A thread of execution *makes progress* when an execution step occurs or a lock-free execution does not complete because there are other concurrent threads that are not blocked in a standard library function (see above).**
>
> **For a thread of execution providing *concurrent forward progress guarantees*, the implementation ensures that the thread will eventually make progress for as long as it has not terminated. [Note: This is required**

**regardless of whether or not other threads of executions (if any) have been or are making progress. To eventually fulfill this requirement means that this will happen in an unspecified but finite amount of time. --- end note]**

**It is implementation-defined whether the implementation-created thread of execution that executes `main` (3.6.1 [basic.start.main]) and the threads of execution created by `std::thread` (30.3.1 [thread.thread.class]) provide concurrent forward progress guarantees. [Note: General-purpose implementations are encouraged to provide these guarantees. --- end note]**

Change 17.3.2 as follows:

block
~~place a thread in the blocked state~~
**a thread of execution that blocks is waiting for some condition (other than for the implementation to execute its execution steps) to be satisfied before it can continue execution past the blocking operation**

Change 17.3.26 as follows:

unblock
~~place a thread in the unblocked state~~
**satisfy a condition that one or more blocked threads of execution are waiting for**

Remove 17.3.3 because it is not used anymore in 17.3.2 or 17.3.26, nor elsewhere in the standard.

Insert the following paragraph after 29.4p2:

**Atomic operations that are not lock-free are considered to potentially block (see 1.10.2 [intro.progress]).**